
CPEN 455 Final Project

Kelvin Wong

Department of Computer Engineering
University of British Columbia
kelvinhk@student.ubc.ca

Abstract

This paper explores enhancements to the PixelCNN++ generative model through class conditioning and architectural refinements aimed at improving image quality and classification performance. Building on the autoregressive PixelCNN framework, we implement a Conditional PixelCNN++ that incorporates label embeddings, U-Net-style skip connections, and batch normalization to preserve spatial information and stabilize training. The model was trained using an RTX 3090 GPU in a Conda environment, leveraging advanced optimization techniques such as adaptive learning rate scheduling, weight decay, and early stopping. Data augmentation strategies were employed to improve generalization. Through an ablation study, we examine the impact of model complexity and training augmentations on classification accuracy, with the full model achieving an FID score of 30.2 and peak classification accuracy of 30%. While the model effectively captures global image structure, it struggles with rendering fine details and high-level semantic features. We discuss potential improvements, including more expressive conditioning mechanisms and alternative training strategies, to address current limitations and enhance model performance for downstream tasks.

1 Basic Tasks

1.1 Running the Basic Model

I ran the model on Ubuntu, with an RTX 3090. I installed the dependencies using Conda and ran the model using the `run.sh` shell script (Appendix A). On this GPU, it trains at about 24it/s, and takes a few hours to complete training. It created images in the `samples` folder, outputs models in the `models` folder, and logged to Wandb. The models were initially saved every 50 epochs. The Wandb dashboard is shown in Appendix B.

1.2 PixelCNN++ Model Overview

The PixelCNN++ model builds upon the original PixelCNN architecture by introducing several key innovations that enhance its performance and stability. In essence, PixelCNN++ replaces the traditional softmax output with a discretized logistic mixture likelihood, which better captures the dependencies between pixel values and results in smoother, more natural image generation. The model further refines spatial coherence by adopting a two-stream processing approach that separately handles vertical and horizontal dependencies, and it leverages a coarse-to-fine hierarchical strategy with downsampling to effectively capture image structure at multiple resolutions.

2 Model

The PixelCNN architecture generates images using an autoregressive approach, meaning each pixel is generated sequentially based on previously generated pixels. This is primarily built using masked convolutions to ensure that the model adheres to the autoregressive properties. However, the standard PixelCNN lacks any conditioning context, which limits its applicability for tasks that require class-specific generation.

2.1 Differences Between PixelCNN and Conditional PixelCNN++

2.1.1 Conditioning Mechanism

The standard PixelCNN operates on raw pixel data, while the Conditional PixelCNN++ incorporates a class conditioning mechanism. Mathematically for an image x and a class label c , the conditional probability of the image can be expressed as:

$$P(x|c) = \prod_{i=1}^N P(x_i | x_{<i}, c)$$

In my implementation, the class labels are mapped into a continuous space using an embedding layer:

$$e_c = \text{Embedding}(c)$$

where e_c is the embedding vector for class c . This is done in the code as follows:

```
self.embedding = nn.Embedding(self.NUM_CLASSES, self.nr_filters)
```

2.1.2 U-Net Architecture

Standard PixelCNN employs a straightforward encoder-decoder structure whereas Conditional PixelCNN++ utilizes a U-Net architecture, which includes multiple fusion points for class information. My implementation uses both early, middle and late fusion. Early fusion processes the class labels through an embedding layer and linear transformations. The middle layer injects class information at multiple points in the encoder, and the late fusion adds class information in the decoder path (Appendix B). The U-Net architecture enhances feature propagation through both the contracting and expanding paths. Skip Connections: For a layer in the expanding path receiving input from the corresponding layer in the contracting path, the operation can be defined as:

$$y = g(h) + f(x_c)$$

where $g(h)$ is the feature map from the current layer in the decoder, and $f(x_c)$ is the feature map from the encoder corresponding to the same spatial resolution. This addition can help preserve spatial information that would otherwise be lost due to down-sampling. Residual Learning: The gated convolutions utilized in PixelCNN++ can also be framed using residual learning concepts:

$$z = \text{Conv}(x) + x$$

This allows the network to learn the difference (residual) from the identity mapping, facilitating deeper networks without incurring significant training difficulties.

2.1.3 Batch Normalization

Conditional PixelCNN++ employs batch normalization at various stages to improve training stability and enhance convergence. By normalizing the activations of each layer using the mean and variance of the mini-batch, batch normalization addresses internal covariate shift, allowing the network to learn more effectively. The transformation applied is given by:

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

where μ_B and σ_B^2 are the mean and variance of the mini-batch, and ϵ is a small constant added for numerical stability. After normalization, the output is scaled and shifted via learned parameters γ and β :

$$y = \gamma\hat{x} + \beta$$

where γ and β are learnable parameters. This inclusion of batch normalization within the Conditional PixelCNN++ architecture helps maintain consistent feature distributions across training iterations, improving the model’s ability to generate high-quality images and perform accurate classifications.

3 Experiments

3.1 Training Methodology

To achieve optimal model performance, we implemented a comprehensive training strategy incorporating multiple optimization techniques and regularization methods. At the core of our approach, we utilized the Adam optimizer with carefully tuned parameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$) and a weight decay of $1e-5$ to prevent overfitting. The learning rate management was handled through a sophisticated adaptive scheme, starting with an initial rate of $1e-4$ and employing a ReduceLROnPlateau scheduler that decreased the rate by a factor of 0.5 when validation performance plateaued, while maintaining a minimum threshold of $1e-6$.

Data augmentation played a crucial role in improving our model’s generalization capabilities (Appendix D). Our augmentation pipeline included:

- Random horizontal flips with 50% probability
- Color jittering with precise parameter tuning (brightness, contrast, saturation, and hue variations of 0.05)
- Consistent image resizing to 32×32 pixels

To ensure training stability and prevent overfitting, we implemented several critical techniques. Batch normalization was employed throughout the network to maintain stable training dynamics, while gradient norm clipping with a maximum threshold of 1.0 was applied to prevent exploding gradients. We performed adaptive hyperparameter tuning for model complexity, focusing particularly on parameters such as `nr_filter` and `nr_resnet`. The training process was closely monitored through validation error tracking, which served as our primary indicator of model performance.

Our early stopping strategy utilized a patient mechanism with a 15-epoch patience window, carefully monitoring validation accuracy for improvements. Training would automatically terminate if no improvement was observed after this period, striking a balance between adequate training time and overfitting prevention. Throughout the training process, we maintained a robust model checkpointing system, saving the best-performing model based on validation accuracy to ensure we preserved the optimal model state.

3.2 Ablation Study

Below are our ablation experiment results. Generated images, FID, and validation graphs can be found in Appendix E and F.

Table 1: Model Complexity Ablation Results

Configuration	Accuracy (%)
Base model	25
With increased <code>nr_filter</code>	28
With increased <code>nr_resnet</code>	30

Table 2: Training Enhancement Ablation Results

Configuration	Accuracy (%)
Without batch normalization	22
Without data augmentation	24
Full model with all enhancements	30

3.3 Quantitative and Qualitative Analysis

Our model achieved a Fréchet Inception Distance (FID) score of 30.2. The classification accuracy peaked at approximately 30%, which, while modest, provides a baseline for this specific task. These metrics serve as quantitative indicators of our model’s current capabilities and highlight areas for potential future improvements.

For generative tasks, there was strong structural coherence in generated images as well as having coherence between the different classes. However, the images that were generated were difficult to understand to a human, but that is because our loss function was based on FID rather than another processes such as reinforcement learning with human feedback.

The largest limitation of our model is the low classification accuracy. We tried to increase the classification accuracy by increasing model complexity, as noted above, but they were all ultimately unsuccessful. The 30% accuracy rate, considered alongside the qualitative analysis, suggests that while the model can capture broad structural patterns, it faces challenges in consistently generating highly detailed content.

4 Conclusion

Our experiments revealed that while the model achieved moderate performance with an FID score of 30.2 and classification accuracy of 30%, there were clear trade-offs between model complexity and performance. The model demonstrated strong capabilities in maintaining global image structure but struggled with fine details and text legibility, particularly in complex scenes. Data augmentation techniques proved essential in improving both generation quality and classification performance. Given additional time and resources, future work could explore more sophisticated conditioning mechanisms, such as incorporating MLPs for processing label embeddings to potentially enhance class conditioning. Furthermore, investigating alternative training strategies and conducting more extensive hyperparameter optimization, particularly focusing on learning rate schedules and batch size configurations, could yield improved results. These modifications could potentially address the current limitations in text rendering and fine detail generation while maintaining the model’s strengths in global structure preservation.

5 References

- [1] Salimans, T., Karpathy, A., Chen, X. & Kingma, D.P. (2017) PixelCNN++: Improving the PixelCNN with Discretized Logistic Mixture Likelihood. *arXiv preprint arXiv:1701.05517*.
- [2] Perez, L. & Wang, J. (2017) The Effectiveness of Data Augmentation in Image Classification using Deep Learning. *arXiv preprint arXiv:1712.04621*.
- [3] Yassin. (2023) Adam vs. AdamW: Understanding Weight Decay and its Impact on Model Performance. *Medium*. Available at: <https://yassin01.medium.com/adam-vs-adamw-understanding-weight-decay-and-its-impact-on-model-performance-b7414f0af8a1>
- [4] Liao, R. (2025) *CPEN 455/555: Deep Learning Lecture Slides*. University of British Columbia. Available at: <https://lrjconan.github.io/UBC-CPEN455-DL/>

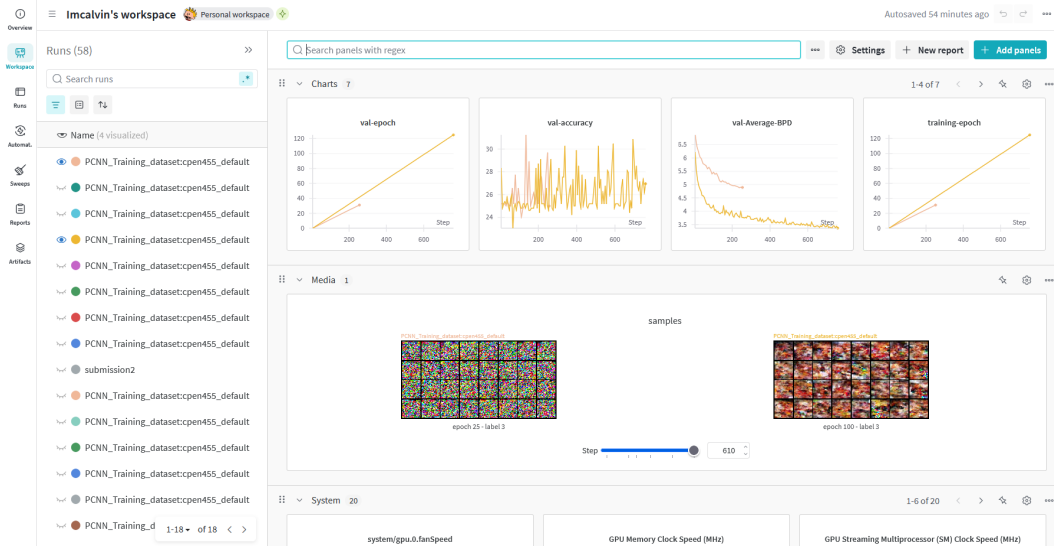
A Appendix / Run.sh Script

```

100% | 20/91.2M [00:01:00:00, 82.00B/s]
100% | 27/27 [00:02:00:00, 13.18t/s]
100% | 17/17 [00:01:00:00, 12.42t/s]
100% | 17/17 [00:01:00:00, 14.21t/s]
100% | 129/129 [01:15:00:00, 1.72t/s]
100% | 16/16 [00:09:00:00, 1.62t/s]
100% | 16/16 [00:09:00:00, 1.63t/s]
Epoch 1:
Train - Accuracy: 0.00%, Loss: 6.9212
Val - Accuracy: 25.20%, Loss: 6.4385
100% | 129/129 [01:11:00:00, 1.81t/s]
100% | 16/16 [00:09:00:00, 1.63t/s]
Epoch 2:
Train - Accuracy: 0.00%, Loss: 6.6823
Val - Accuracy: 25.20%, Loss: 6.2468
100% | 129/129 [01:09:00:00, 1.86t/s]
100% | 16/16 [00:08:00:00, 1.78t/s]
Epoch 3:
Train - Accuracy: 0.00%, Loss: 6.4250
Val - Accuracy: 25.80%, Loss: 6.0375
100% | 129/129 [01:09:00:00, 1.85t/s]
100% | 16/16 [00:08:00:00, 1.77t/s]
Epoch 4:
Train - Accuracy: 0.00%, Loss: 6.2585
Val - Accuracy: 25.39%, Loss: 5.9619
New best model saved with validation accuracy: 25.39%
100% | 129/129 [01:09:00:00, 1.85t/s]
100% | 16/16 [00:09:00:00, 1.67t/s]
Epoch 5:
Train - Accuracy: 0.00%, Loss: 6.1552
Val - Accuracy: 25.80%, Loss: 5.7593
100% | 129/129 [01:09:00:00, 1.86t/s]
100% | 16/16 [00:09:00:00, 1.77t/s]
Epoch 6:
Train - Accuracy: 0.00%, Loss: 6.0231
Val - Accuracy: 25.80%, Loss: 5.7289
100% | 129/129 [01:09:00:00, 1.86t/s]
100% | 16/16 [00:09:00:00, 1.77t/s]
Epoch 7:
Train - Accuracy: 0.00%, Loss: 5.9729
Val - Accuracy: 26.56%, Loss: 5.7289
New best model saved with validation accuracy: 26.56%
100% | 129/129 [01:09:00:00, 1.86t/s]
100% | 16/16 [00:09:00:00, 1.79t/s]
2% | 8/500 [20:17:12:55:31, 94.58s/t]
81% | 104/129 [00:57:00:14, 1.73t/s]

```

B Appendix / Wandb Dashboard



C Appendix / Early, Middle, Late Fusion Code

```

# In __init__:
self.embedding = nn.Embedding(self.NUM_CLASSES, self.nr_filters)
self.emb_linear = nn.Sequential(
    nn.Linear(self.nr_filters, self.nr_filters),
    nn.BatchNorm1d(self.nr_filters),
    nn.ELU(),
    nn.Linear(self.nr_filters, self.nr_filters),
    nn.BatchNorm1d(self.nr_filters)
)

# In forward:
label_embeddings = self.embedding(class_cond.to(x.device))
label_embeddings = self.emb_linear(label_embeddings)

```

```

label_embeddings = label_embeddings.unsqueeze(-1).unsqueeze(-1)

# In __init__:
self.middle_fusion = nn.Sequential(
    nn.Linear(self.nr_filters, self.nr_filters),
    nn.BatchNorm1d(self.nr_filters),
    nn.ELU(),
    nn.Linear(self.nr_filters, self.nr_filters),
    nn.BatchNorm1d(self.nr_filters)
)

# In forward:
if i in [0, 1]: # Add in multiple layers
    mid_emb = self.middle_fusion(label_embeddings.squeeze(-1).squeeze(-1))
    mid_emb = mid_emb.unsqueeze(-1).unsqueeze(-1)
    self.add_embedding_to_u_ul(u_list, mid_emb)
    self.add_embedding_to_u_ul(ul_list, mid_emb)

# In __init__:
self.late_fusion = nn.Sequential(
    nn.Linear(self.nr_filters, self.nr_filters),
    nn.BatchNorm1d(self.nr_filters),
    nn.ELU(),
    nn.Linear(self.nr_filters, self.nr_filters),
    nn.BatchNorm1d(self.nr_filters)
)

# In forward:
if i in [1, 2]: # Add in multiple layers
    late_emb = self.late_fusion(label_embeddings.squeeze(-1).squeeze(-1))
    late_emb = late_emb.unsqueeze(-1).unsqueeze(-1)
    u += late_emb
    ul += late_emb

def add_embedding_to_u_ul(self, tensor_list, embedding_tensor):
    for i in range(len(tensor_list)):
        tensor_list[i] += embedding_tensor

```

D Appendix / Data Augmentation

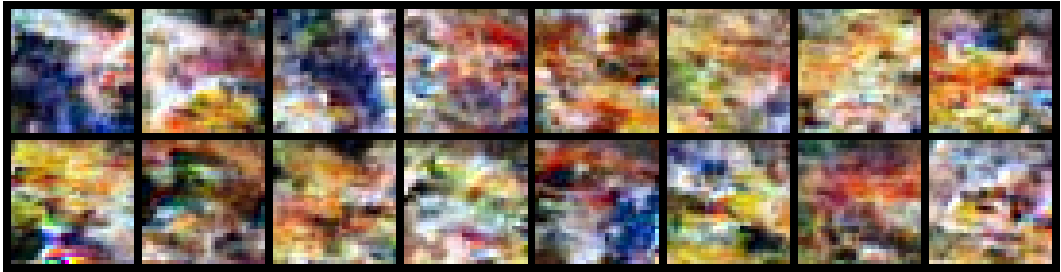
```

ds_transforms_train = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ColorJitter(brightness=0.05, contrast=0.05, saturation=0.05, hue=0.05),
    transforms.Resize((32, 32)),
    rescaling
])

```

E Appendix / Generated Images For Ablation Study

Generated images from the model. The first image is the baseline model; the middle one is increased with `nr_filter`, the last one is increased with `nr_resnet`.





F Appendix / FID and Validation Graph

